Introduction to Arduino and Raspberry Pi Fundamentals

Exam-Oriented Question Bank & Complete Answer Key

PART 1: QUESTION BANK OVERVIEW

10-Mark Questions (Long Answer)

1. Explain the complete hardware architecture, technical specifications, and internal components of the Arduino UNO development board.
2. Describe the step-by-step procedure for installing the Arduino IDE, establishing hardware connections, configuring board parameters, and verifying the environment with a test sketch.
3. Detail the architectural block diagram, internal components, and hardware interfaces of the Raspberry Pi single-board computer.
4. Explain the concept of IoT physical devices, endpoints, and the end-to-end data flow architecture from sensors to cloud applications.
5. Provide a detailed comparative evaluation between the Arduino UNO microcontroller board and the Raspberry Pi single-board computer across hardware, software, processing, and application domains.

5-Mark Questions (Medium Answer)

1. Explain the fundamental structure of an Arduino sketch, highlighting the roles of `setup()` and `loop()` functions.
2. Discuss the various operating systems supported by Raspberry Pi and justify why Linux-based operating systems are preferred for IoT deployments.
3. Outline the hardware prerequisites and execution steps required to flash and configure Raspberry Pi OS using an SD card.
4. Describe the primary digital and analog I/O control functions in the Arduino programming environment.
5. Explain the function and significance of General Purpose Input/Output (GPIO) pins on Arduino UNO and Raspberry Pi boards.
6. Distinguish between an IoT physical device and an IoT endpoint, illustrating with a multi-endpoint scenario.
7. Detail the use of variables, control structures (`if-else`, `for` loop), and custom functions in Arduino programming.
8. Describe the functional modules available within the Raspberry Pi Configuration Tool across System, Interfaces, Performance, and Localization tabs.

2-Mark Questions (Short Answer)

9. Define an open-source electronics platform in the context of Arduino.
10. State the core role of the ATmega328P microcontroller on the Arduino UNO board.
11. What is the purpose of the 16 MHz crystal oscillator on the Arduino UNO board?
12. Differentiate between standard Digital I/O pins and Pulse Width Modulation (PWM) pins on Arduino UNO.
13. Explain the primary function of the onboard voltage regulator on Arduino UNO.
14. Define sensors and actuators in an IoT hardware ecosystem with relevant examples.
15. Define a Single-Board Computer (SBC) in the context of Raspberry Pi.
16. What is the functional requirement of the MicroSD card slot in Raspberry Pi hardware?
17. Differentiate between Camera Serial Interface (CSI) and Display Serial Interface (DSI) on Raspberry Pi.
18. What is the primary role of an IoT gateway in end-to-end data processing?

PART 2: COMPLETE ANSWER KEY

10-MARK QUESTIONS & ANSWERS

Q1. Explain the complete hardware architecture, technical specifications, and internal components of the Arduino UNO development board. [10 Marks]

Definition & Overview

The Arduino UNO is an open-source microcontroller development board built around the 8-bit Microchip ATmega328P. It is the standard platform for prototyping, interfacing sensors and actuators, and running low-power real-time embedded control loops.

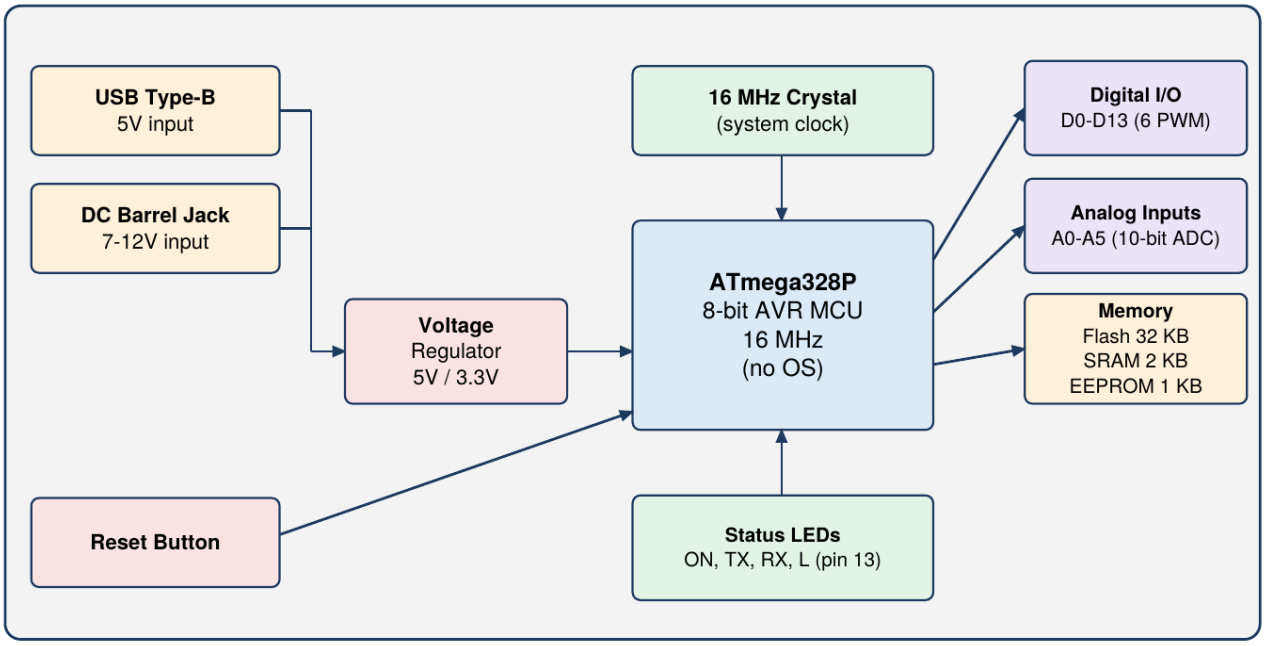


Fig 1: Arduino UNO hardware block diagram

- **Core Microcontroller (ATmega328P):** Acts as the CPU and brain of the board: it executes user code, processes sensor signals and drives actuators. It runs at 16 MHz and executes the sketch directly without any operating system.
- **Power Supply (USB and DC Jack):** The board is powered through USB Type-B (5V) or the DC barrel jack (7-12V), and it automatically selects the active source. External batteries connect to the jack or the VIN pin.
- **Onboard Voltage Regulation:** A linear regulator steps higher input voltages (9V or 12V) down to a stable 5V for the MCU and sensors. An auxiliary 3.3V rail supplies low-voltage sensor modules.
- **Memory (Flash, SRAM, EEPROM):** The ATmega328P has 32 KB Flash (0.5 KB reserved for the bootloader) to store sketches, 2 KB SRAM for runtime variables and 1 KB EEPROM for data that must survive power cycles.
- **Clock Generation (16 MHz Crystal):** A quartz crystal supplies the clock signal for CPU instruction timing. Functions such as `delay(1000)` depend on it to pause for exactly one second.
- **Digital I/O Pins (0-13):** 14 pins read or drive binary HIGH (5V) / LOW (0V) signals and are set as INPUT or OUTPUT in software. Pins 0 (RX) and 1 (TX) also act as hardware serial lines.
- **PWM Capability:** Pins 3, 5, 6, 9, 10 and 11 support hardware Pulse Width Modulation, which imitates analog output by varying the duty cycle. It is used for LED brightness and motor speed control.
- **Analog Input Header (A0-A5):** Six analog pins feed an internal 10-bit ADC that maps 0-5V to integer values 0-1023. They read continuous sensors such as potentiometers and light sensors.
- **Status and Diagnostic LEDs:** The ON LED shows power, the TX/RX LEDs flash during USB transfers, and the L LED on pin 13 gives a quick check that the program is running.
- **Reset Button and Bootloader:** The reset button pulls the reset line LOW and restarts the sketch without removing power. The bootloader lets code be uploaded over USB without an external programmer.

Q2. Describe the step-by-step procedure for installing the Arduino IDE, establishing hardware connections, configuring board parameters, and verifying the environment with a test sketch. [10 Marks]

Definition & Overview

Setting up the Arduino ecosystem means installing the IDE, connecting the board over USB, configuring the board and port in software, and compiling a verification sketch.

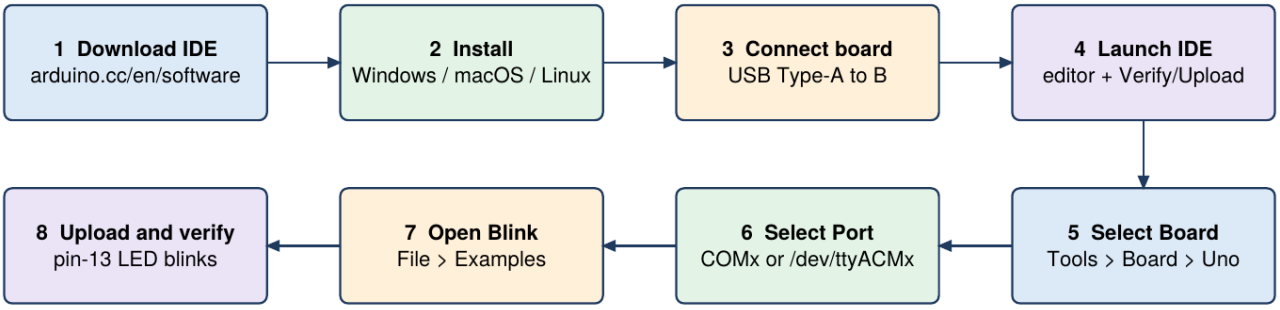


Fig 2: Arduino IDE setup and verification flow

- **Downloading the IDE:** Get the official installer from arduino.cc/en/software and pick the package for the host OS (Windows, macOS or Linux). On Windows you can choose the executable installer or a portable ZIP.
- **Installing on Windows:** Run the `.exe`, accept the licence, keep the default components and choose the install folder. Keep the USB driver option ticked so the PC recognises the board.
- **Installing on macOS and Linux:** On macOS open the `.dmg` and drag the Arduino IDE icon to Applications. On Linux extract the `.tar.xz` archive and run `install.sh` from the terminal.
- **Connecting the Hardware:** Join the UNO to the computer with a USB Type-A to Type-B cable, which also supplies 5V power. The green ON LED confirms the connection.
- **Launching the IDE:** The IDE has a text editor, a toolbar with Verify and Upload buttons, and a console output window at the bottom.
- **Target Board Selection:** Go to `Tools > Board` and select Arduino Uno so the code is compiled for the correct hardware.
- **Serial Port Selection:** Choose the active path under `Tools > Port`: a COM port such as COM3 on Windows, or `/dev/ttyUSB0` or `/dev/ttyACM0` on Linux.
- **Loading the Blink Sketch:** Open `File > Examples > 01.Basics > Blink` to load the ready-made LED blinking program as the test project.
- **Compile and Upload:** Click the Upload (right arrow) button. The IDE compiles the C/C++ sketch with GCC into machine code and writes it to the ATmega328P Flash over USB.
- **Hardware Verification:** The TX and RX LEDs flash rapidly during upload. When it finishes, the on-board LED on pin 13 blinks at 1-second intervals.

Q3. Detail the architectural block diagram, internal components, and hardware interfaces of the Raspberry Pi single-board computer. [10 Marks]

Definition & Overview

The Raspberry Pi is a low-cost, credit-card-sized Single-Board Computer (SBC) from the Raspberry Pi Foundation. Unlike a microcontroller it contains a full System-on-Chip and can run a complete Linux operating system and complex software.

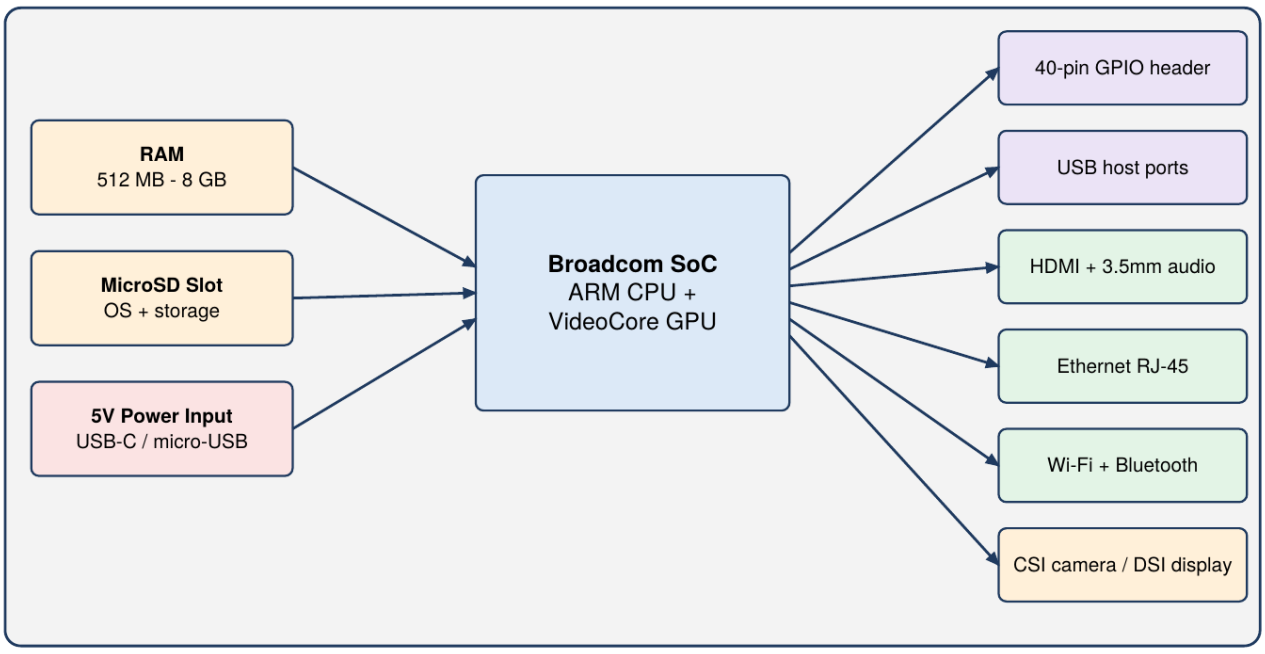


Fig 3: Raspberry Pi architectural block diagram

- **System-on-Chip (SoC):** A Broadcom SoC (e.g., BCM2835) integrates an ARM CPU with a VideoCore GPU. It handles execution, data processing, graphics and peripheral interfacing.

- **Onboard RAM:** Volatile RAM from 512 MB up to 8 GB (depending on model) supports multitasking, resource-heavy applications and the Linux desktop.
- **MicroSD Slot (Storage and Boot):** The board has no internal flash for the OS; a MicroSD card flashed with an OS image (e.g., Raspberry Pi OS) acts as the boot device and persistent storage.
- **40-Pin GPIO Header:** Exposes digital input/output lines so Linux software can read sensors, drive relays and motors, and use SPI, I2C and UART protocols.
- **HDMI Output:** Full-size or micro HDMI ports carry digital video and audio, turning the Pi into a desktop workstation or media centre.
- **USB Ports:** Multiple USB host ports connect keyboards, mice, storage drives, network adapters and webcams.
- **Ethernet Interface:** Model B boards have an RJ-45 port for reliable wired networking, useful for web servers, cloud communication or acting as an edge gateway.
- **Wi-Fi and Bluetooth:** Modern boards include 2.4/5 GHz Wi-Fi and Bluetooth for remote access (SSH/VNC) and IoT cloud streaming without using USB ports.
- **Camera Serial Interface (CSI):** A ribbon connector that links official camera modules directly to the processor's image pipeline for high-speed capture.
- **Display Serial Interface (DSI) and Audio:** A 15-pin DSI connector drives LCD touchscreens directly, and a 3.5 mm jack outputs analog audio.

Q4. Explain the concept of IoT physical devices, endpoints, and the end-to-end data flow architecture from sensors to cloud applications. [10 Marks]

Definition & Overview

IoT architectures bridge physical environments and computing networks. Data flows from sensing hardware (devices and endpoints) through gateways to centralised cloud applications, and commands flow back down.

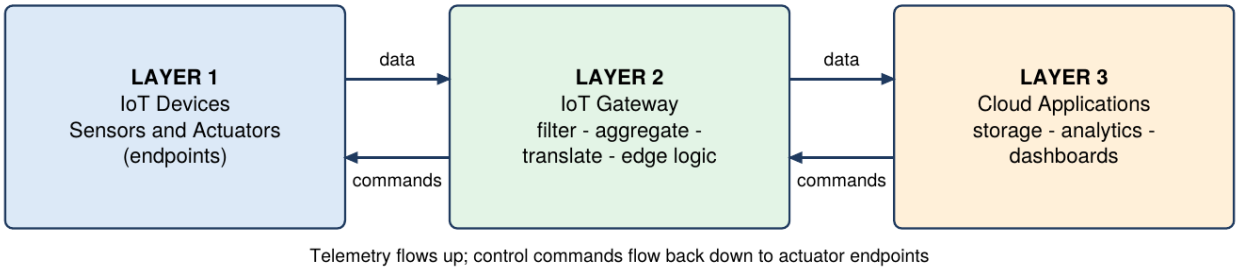


Fig 4: End-to-end IoT data flow (Devices - Gateway - Applications)

- **Physical Devices:** Hardware units deployed in the environment that capture environmental parameters or turn electrical signals into mechanical action.
- **IoT Endpoints:** An endpoint is a network-connected entity (hardware or software address) that sends or receives data packets. The device is the hardware unit; the endpoint is the network entity.
- **Sensor Endpoints:** Measure temperature, humidity, light or gas levels and convert them to digital signals. They are pure data sources that send telemetry upward.
- **Actuator Endpoints:** Receive digital commands over the network and trigger actions, e.g., smart relays, solenoid valves or servo motors.
- **Hybrid Endpoints:** Combine sensing and actuation in one node. A smart thermostat senses room temperature and switches the HVAC relay locally.
- **Multi-Endpoint Scenario:** One physical device can host several logical endpoints. A Raspberry Pi with temperature, humidity and motion sensors can expose `/temp`, `/humidity` and `/motion` REST endpoints.
- **Layer 1: Sensing and Actuation:** Sensors sample conditions such as ambient temperature and pass raw signals to microcontrollers, which digitise them.
- **Layer 2: Gateways (Edge Processing):** Gateways bridge low-power devices and remote servers: they collect telemetry from many endpoints, aggregate, filter noise and reformat payloads before sending over the WAN.
- **Edge Computation and Latency:** Gateways also check local thresholds, so they can trigger fast safety shutdowns or drop redundant samples, saving cloud bandwidth.
- **Layer 3: Cloud Applications:** Applications store telemetry in databases, show dashboards, analyse trends and send notifications or commands back to actuator endpoints.

Q5. Provide a detailed comparative evaluation between the Arduino UNO microcontroller board and the Raspberry Pi single-board computer across hardware, software, processing, and application domains. [10 Marks]

Definition & Overview

Both are popular open-source hardware platforms but belong to different categories: the Arduino UNO is a real-time 8-bit microcontroller board, while the Raspberry Pi is a full 32/64-bit Single-Board Computer.

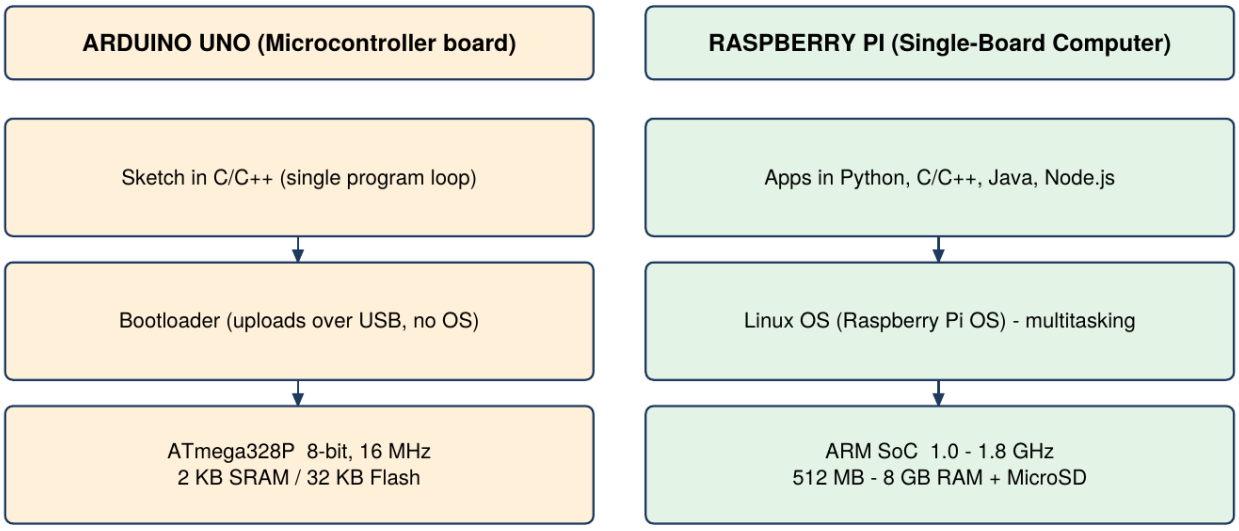


Fig 5: Layered architecture - Arduino UNO vs Raspberry Pi

Feature	Arduino UNO	Raspberry Pi
Device Classification	Microcontroller development board	Single-Board Computer (SBC)
Processor	8-bit Microchip ATmega328P	32/64-bit ARM Cortex (SoC)
Operating System	Bare-metal (no OS)	Full Linux OS (Raspberry Pi OS)
Clock Speed	16 MHz	1.0 GHz to 1.8 GHz
RAM	2 KB SRAM	512 MB to 8 GB
Storage	32 KB Flash	MicroSD card (up to 128 GB+)
GPIO	14 digital (6 PWM) + 6 analog	40 digital GPIO pins
Network	Needs Wi-Fi/Ethernet shield	Onboard Wi-Fi, Bluetooth, Ethernet
Multitasking	Single program loop	Preemptive multitasking OS
Power	Very low (~0.2 W)	Moderate (2 to 6 W)

- **Processing and Speed:** The ATmega328P runs one compiled sketch directly on hardware at 16 MHz, whereas the Pi's ARM processor at 1.0-1.8 GHz gives orders of magnitude more throughput.
- **Operating System:** Arduino code compiles straight to machine code and manages registers natively. The Pi boots Linux, which handles memory, scheduling, file systems and drivers.
- **Memory and Storage:** Arduino's 2 KB RAM and 32 KB Flash limit it to light control loops; the Pi's RAM (up to 8 GB) and MicroSD support databases, streaming and web servers.
- **I/O Capability:** The UNO has 6 analog inputs on a 10-bit ADC for direct sensor reads. The Pi has no internal ADC (needs an external chip) but offers a 40-pin header.
- **Networking:** The Pi has built-in Ethernet, dual-band Wi-Fi and Bluetooth. The UNO needs add-on shields or modules for any network access.
- **Programming Languages:** Arduino uses C/C++ in the Arduino IDE. The Pi supports Python, C, C++, Java, JavaScript (Node.js) and shell scripting.
- **Power Efficiency:** The UNO can run on small batteries or solar cells. The Pi draws 2-6 W and needs a steady 5V/3A supply, making battery-only use harder.
- **Application Domains:** Arduino suits deterministic real-time control (motors, relays, sensor reading). The Pi suits vision, IoT gateways, media centres and web interfaces.
- **Real-World Contrast:** Arduino: read a soil-moisture sensor and switch a pump relay. Raspberry Pi: capture camera frames, detect plant disease, log to SQL and sync reports to a cloud dashboard.

5-MARK QUESTIONS & ANSWERS

Q6. Explain the fundamental structure of an Arduino sketch, highlighting the roles of `setup()` and `loop()` functions. [5 Marks]

Definition & Overview

Programs written for Arduino are called sketches. Every sketch must contain two C/C++ functions: `setup()` and `loop()`.

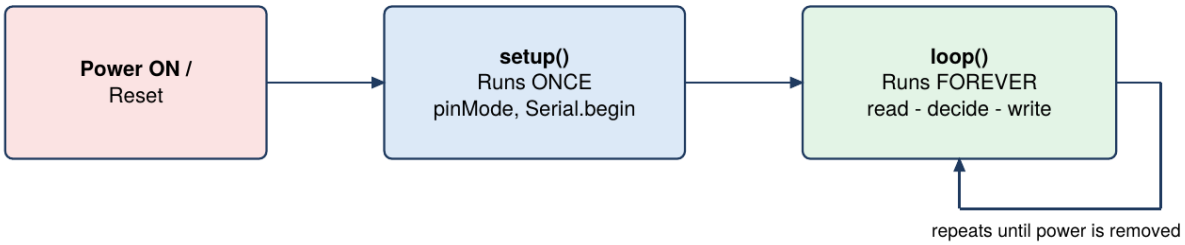


Fig 6: Arduino sketch execution flow

```
void setup() {
  pinMode(13, OUTPUT);     // runs ONCE at start-up / reset
}

void loop() {
  digitalWrite(13, HIGH);  // runs REPEATEDLY
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

- **`setup()` Function:** Runs once when the board powers up or resets. It sets pin modes with `pinMode()`, starts serial communication with `Serial.begin()` and initialises variables.
- **`loop()` Function:** Runs after `setup()` in an infinite cycle. It holds the main logic: sampling inputs, taking decisions and driving outputs until power is removed.
- **Execution Contrast:** `setup()` executes exactly one time at boot, whereas `loop()` repeats endlessly so the microcontroller keeps reacting to the environment.
- **Hardware Initialisation:** Pin directions such as `pinMode(13, OUTPUT);` belong in `setup()`. Placing them in `loop()` would repeat needless initialisation every cycle.
- **Main Application Logic:** Inside `loop()` code reads data with `digitalRead()` or `analogRead()`, processes conditions and updates outputs with `digitalWrite()` or `analogWrite()`, with `delay()` controlling timing.

Q7. Discuss the various operating systems supported by Raspberry Pi and justify why Linux-based operating systems are preferred for IoT deployments. [5 Marks]

Definition & Overview

Unlike a microcontroller, the Raspberry Pi needs a full Operating System to manage hardware, run applications, handle I/O and execute network stacks.

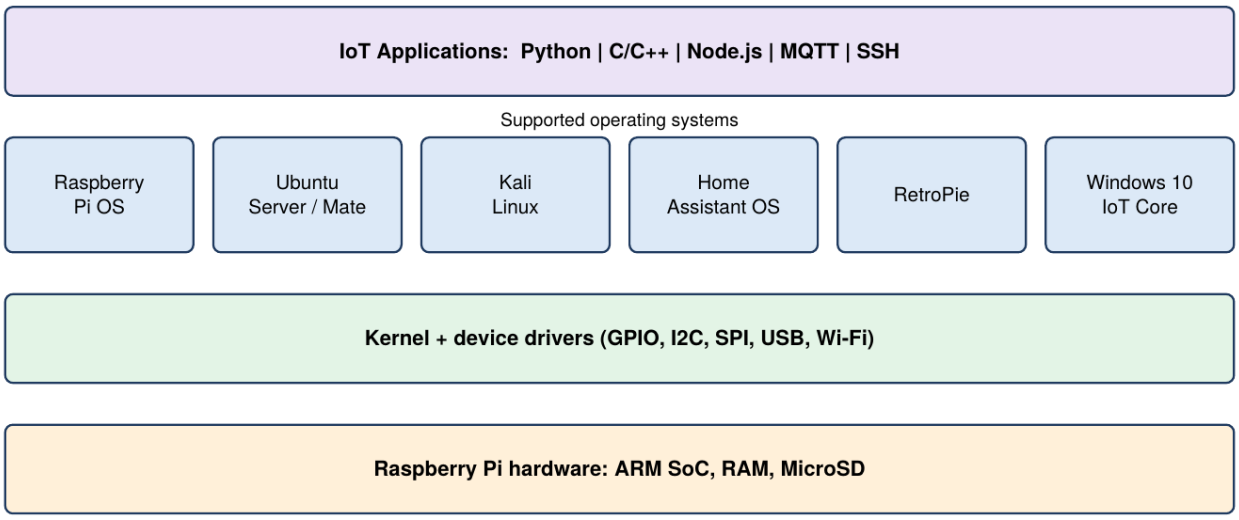


Fig 7: Operating system options on Raspberry Pi

- **Raspberry Pi OS (Raspbian):** The official Debian-based OS tuned for Pi hardware. It ships with Python, Scratch and C/C++ tools, in a full Desktop version and a lightweight command-line Lite version.
- **Ubuntu Server / Ubuntu Mate:** Canonical's builds for the Pi: Ubuntu Server for headless gateways and web servers, Ubuntu Mate for a desktop development environment.
- **Specialised Operating Systems:** Kali Linux (security testing), Windows 10 IoT Core (Visual Studio workflows), RetroPie (emulation) and Home Assistant OS (smart home hubs).
- **Why Linux: Open Source and Lightweight:** Linux is open and modular, so unneeded graphics drivers and services can be removed. Builds like Raspberry Pi OS Lite run well in minimal RAM.

- **Why Linux: Security, Stability and Languages:** Multi-user permissions, SSH remote management and process isolation give stable 24/7 operation. It natively supports Python, C, C++, Java and JavaScript libraries for IoT software.

Q8. Outline the hardware prerequisites and execution steps required to flash and configure Raspberry Pi OS using an SD card. [5 Marks]

Definition & Overview

Preparing a Raspberry Pi means writing the OS image onto a MicroSD card on a host computer and completing the first-boot configuration.

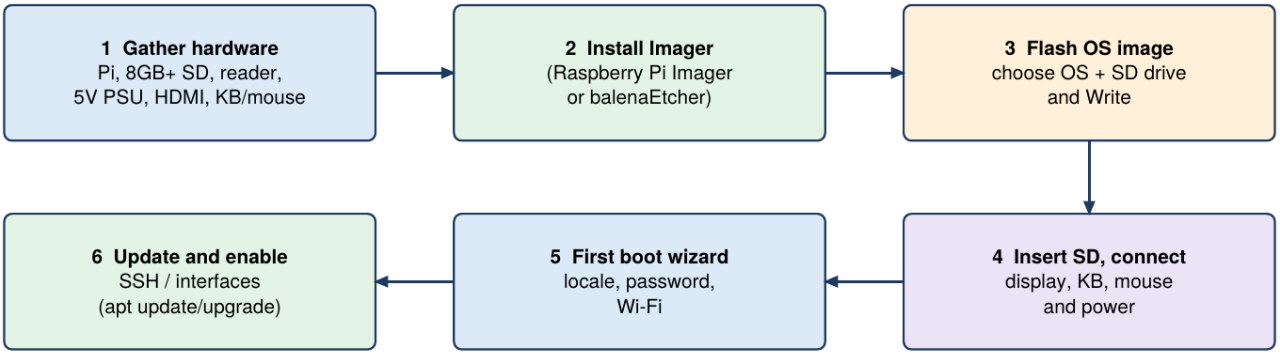


Fig 8: Flashing and configuring Raspberry Pi OS

- **Hardware Prerequisites:** A Raspberry Pi board, a Class 10 MicroSD card (8 GB or more), a card reader for the host computer, a 5V power supply, an HDMI monitor, keyboard and mouse.
- **Flashing Software:** Install an imaging tool on the host. The official choice is Raspberry Pi Imager; balenaEtcher also works.
- **Flashing the Card:** Insert the card, open Raspberry Pi Imager, select the OS (e.g., Raspberry Pi OS 32-bit), select the SD drive and click Write. This formats the card and creates a bootable partition.
- **Assembly and First Boot:** Place the card in the Pi, connect display, keyboard, mouse and the 5V cable. The bootloader reads the SD card and starts the desktop.
- **First-Boot Configuration:** The wizard sets country, language and timezone, creates a secure password (older images used default user `pi` / password `raspberry`), connects Wi-Fi, updates packages with `sudo apt update && sudo apt upgrade`, and optionally enables SSH.

Q9. Describe the primary digital and analog I/O control functions in the Arduino programming environment. [5 Marks]

Definition & Overview

The Arduino core library provides high-level C/C++ functions to read and write digital and analog signals without manually configuring microcontroller registers.

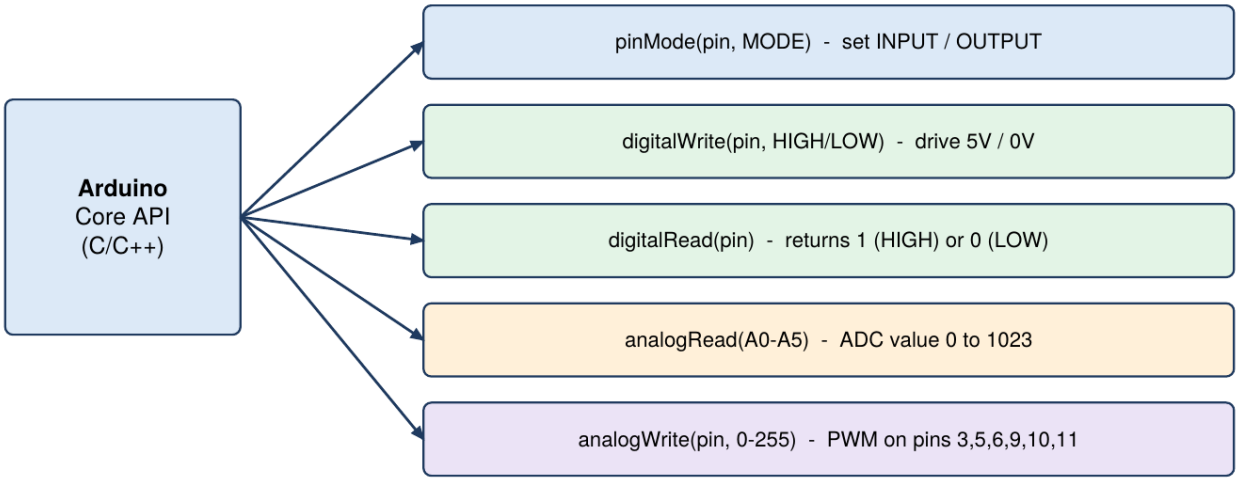


Fig 9: Arduino digital and analog I/O functions

- **`pinMode(pin, MODE)`:** Configures a digital pin as INPUT or OUTPUT, usually inside `setup()`. Example: `pinMode(13, OUTPUT);` prepares pin 13 to drive an LED.
- **`digitalWrite(pin, VALUE)`:** Writes HIGH (5V) or LOW (0V) to an output pin. `digitalWrite(13, HIGH);` turns the attached LED ON.
- **`digitalRead(pin)`:** Reads a digital input and returns 1 (HIGH) or 0 (LOW). Example: `int buttonState = digitalRead(2);` checks a push button.

- **`analogRead(pin)`:** Reads the voltage on A0-A5; the ADC maps 0-5V to 0-1023. Example: `int sensorVal = analogRead(A0);`.
- **`analogWrite(pin, VALUE)`:** Outputs a PWM signal on pins 3, 5, 6, 9, 10, 11 with duty cycle 0 (0V) to 255 (5V). `analogWrite(6, 128);` gives about 50% duty to dim an LED or set motor speed.

Q10. Explain the function and significance of General Purpose Input/Output (GPIO) pins on Arduino UNO and Raspberry Pi boards. [5 Marks]

Definition & Overview

GPIO pins are the electrical interface between a processor and external components, letting software read inputs and drive outputs in the physical world.

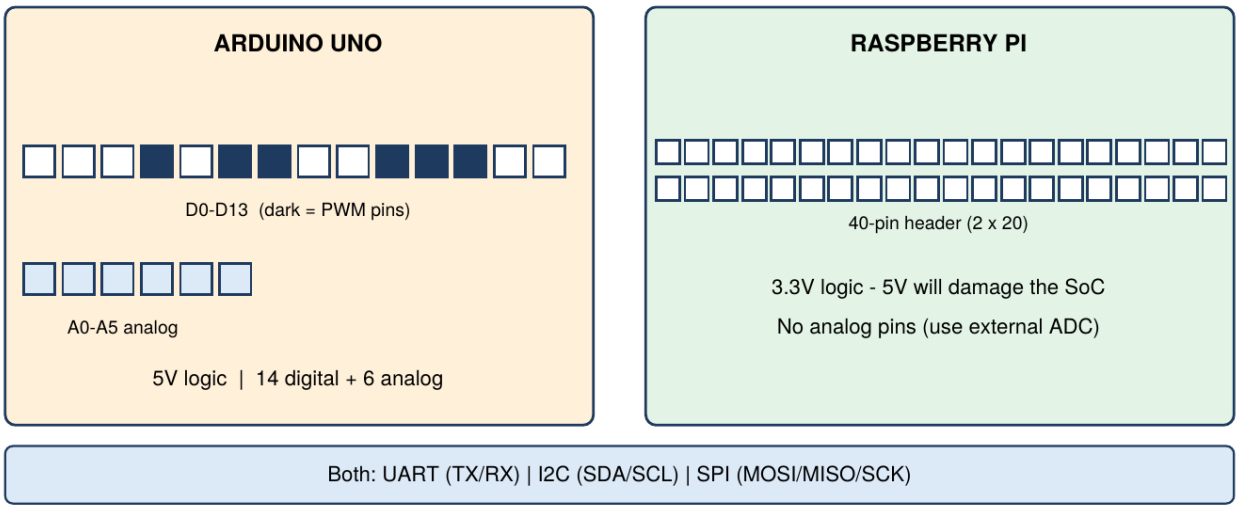


Fig 10: GPIO headers - Arduino UNO (5V) vs Raspberry Pi (3.3V)

- **Fundamental Function:** Each pin is software-configured as input (reads switches and sensors) or output (drives LEDs, relays, buzzers and motors).
- **Arduino UNO Header:** 14 digital pins (6 with PWM) and 6 analog inputs A0-A5. They work at 5V logic, so inputs must not exceed 5V.
- **Raspberry Pi 40-Pin Header:** All GPIO pins operate at 3.3V logic. Applying 5V directly can damage the Broadcom SoC.
- **Communication Protocols:** Designated pins on both boards provide UART (TX/RX), I2C (SDA/SCL) and SPI (MOSI/MISO/SCK) buses in addition to plain ON/OFF switching.
- **Significance Comparison:** Arduino GPIO uses bare-metal commands such as `digitalWrite` for microsecond real-time control. Pi GPIO is controlled via Linux drivers and Python libraries (`RPi.GPIO`), enabling web-triggered and cloud-linked automation.

Q11. Distinguish between an IoT physical device and an IoT endpoint, illustrating with a multi-endpoint scenario. [5 Marks]

Definition & Overview

A physical device is the hardware unit; an endpoint is a network-addressable entity that sends or receives data. The two do not always map one-to-one.

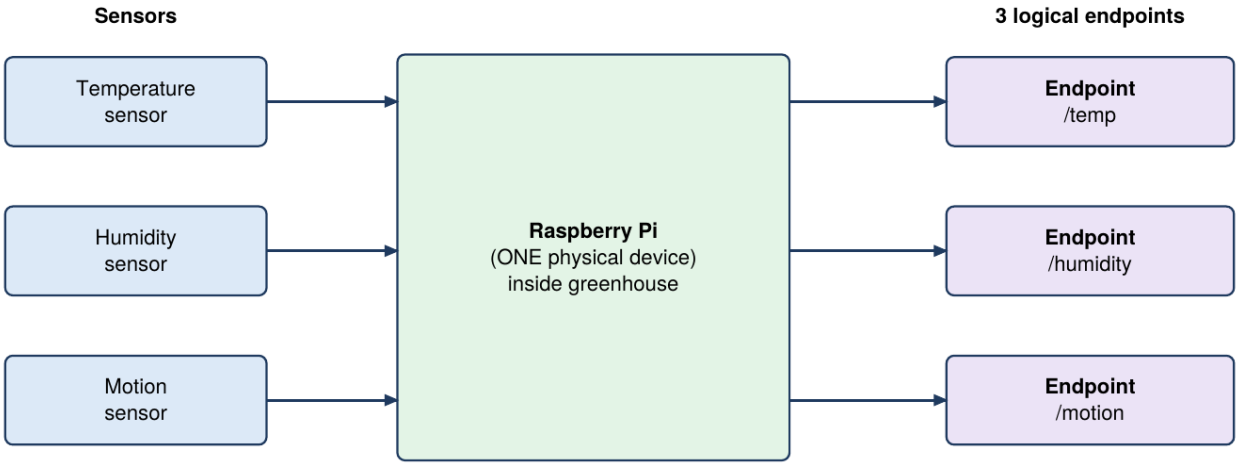


Fig 11: One physical device hosting three logical endpoints

- **IoT Physical Device:** The tangible unit in the field: chassis, circuit board, microcontroller (Arduino/Raspberry Pi), power circuits, sensor modules and wiring.
- **IoT Endpoint:** A network node or web interface that sends telemetry or receives API commands, identified by an IP address, MAC address or RESTful URI.
- **Endpoint Categories:** Sensor endpoints only transmit telemetry, actuator endpoints only receive commands, and hybrid endpoints do both.
- **One-to-Many Mapping:** A basic sensor module may be a single endpoint, but one complex computer can host several software endpoints over one network connection.
- **Illustrative Scenario:** A Raspberry Pi in a greenhouse with temperature, humidity and motion sensors is ONE physical device but exposes THREE endpoints: `/temp` (ambient temperature), `/humidity` (moisture level) and `/motion` (security alerts).

Q12. Detail the use of variables, control structures (`if-else`, `for` loop), and custom functions in Arduino programming. [5 Marks]

Definition & Overview

Arduino code is C/C++. Variables hold state, control structures direct execution flow, and functions group reusable logic.

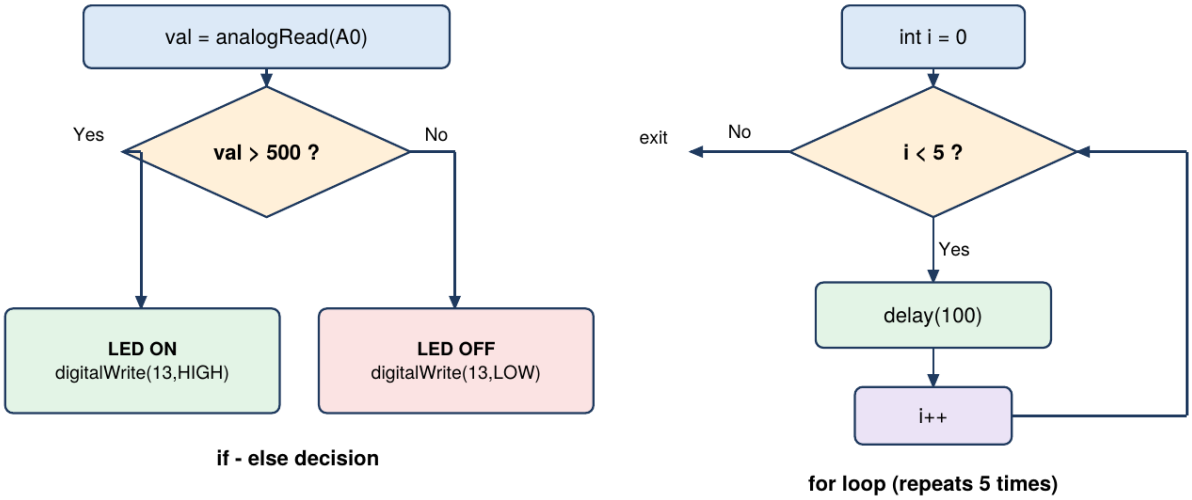


Fig 12: Flow of if-else and for loop

```
int sensorPin = A0;                    // variable

int calculateAverage(int a, int b) {   // custom function
  return (a + b) / 2;
}

void setup() { pinMode(13, OUTPUT); }

void loop() {
  int val = analogRead(sensorPin);
  if (val > 500) { digitalWrite(13, HIGH); }   // if-else
  else           { digitalWrite(13, LOW);  }
  for (int i = 0; i < 5; i++) { delay(100); }  // for loop
}
```

- **Variables and Data Types:** Named memory containers for sensor values, pin numbers and flags. Common types: `int` (16-bit, -32,768 to 32,767), `float`, `char`, `bool` and `String`.
- **`if-else` Structure:** Evaluates a condition: the `if` block runs when true, otherwise the `else` block runs. It is used to compare sensor values with thresholds, e.g., switch a fan on above a temperature limit.
- **`for` Loop:** Repeats a block a fixed number of times using an initial value, a continuation condition and an increment. Example: flash an LED 5 times at start-up.
- **Custom Functions:** Reusable blocks that cut duplication and improve readability. They can take parameters and return a result with `return`.
- **Function Example:** `int addNumbers(int a, int b) { return a + b; }` takes two integers, returns their sum, and the result can be printed with `Serial.println()`.

Q13. Describe the functional modules available within the Raspberry Pi Configuration Tool across System, Interfaces, Performance, and Localization tabs. [5 Marks]

Definition & Overview

The Raspberry Pi Configuration Tool (graphical, or `sudo raspi-config` in the terminal) is the main settings interface of Raspberry Pi OS for enabling interfaces, tuning performance and managing regional options.

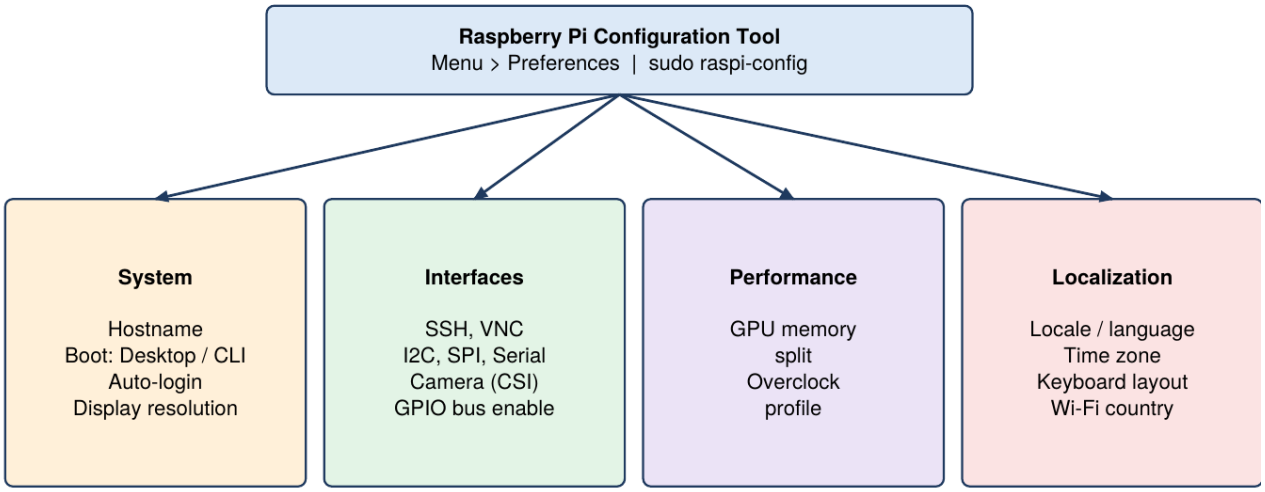


Fig 13: Raspberry Pi Configuration Tool tabs

- **Accessing the Tool:** On the desktop go to `Menu > Preferences > Raspberry Pi Configuration`; in a terminal run `sudo raspi-config`.
- **System Tab:** Change the hostname, choose boot to Desktop GUI or CLI, set auto-login and adjust display resolution.
- **Interfaces Tab:** Enable or disable hardware buses (I2C, SPI, Serial UART), the camera (CSI) and remote access via SSH or VNC.
- **Performance Tab:** Set GPU memory allocation (balancing RAM between GPU and CPU) and CPU overclock profiles for demanding workloads.
- **Localization Tab:** Set locale, language, timezone and keyboard layout so typed characters match the physical keyboard.

2-MARK QUESTIONS & ANSWERS

Q14. Define an open-source electronics platform in the context of Arduino. [2 Marks]

Arduino is an open-source electronics platform built on freely accessible hardware and software designs.

- **Open Hardware and Software:** Schematics, PCB layouts and IDE libraries are free to inspect, modify and manufacture.
- **Accessibility:** Students, makers and professionals can build custom boards and prototype interactive projects at low cost.

Q15. State the core role of the ATmega328P microcontroller on the Arduino UNO board. [2 Marks]

The ATmega328P is the 8-bit AVR microcontroller that serves as the main processor of the Arduino UNO.

- **Central Processing Brain:** Executes the binary instructions compiled from the user's C/C++ sketch.
- **I/O Signal Management:** Reads sensor inputs, processes logic and drives outputs to actuators through the pin headers.

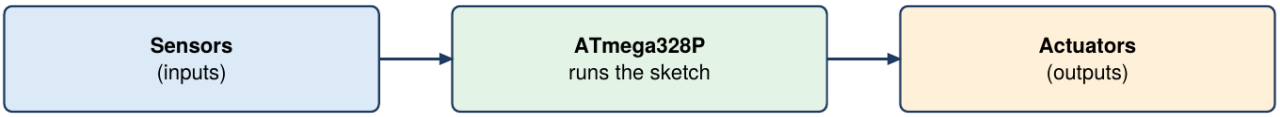


Fig 14: Role of the ATmega328P

Q16. What is the purpose of the 16 MHz crystal oscillator on the Arduino UNO board? [2 Marks]

The 16 MHz crystal oscillator is the timing component of the Arduino UNO.

- **System Clock Generation:** Produces a stable 16 MHz clock that synchronises instruction execution in the ATmega328P.
- **Accurate Time Delays:** Gives the reference for timing functions, so `delay(1000)` pauses for exactly one second.

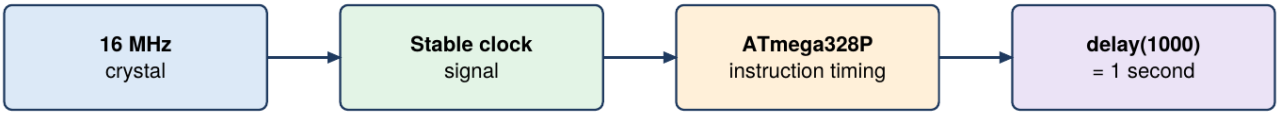


Fig 15: Clock generation

Q17. Differentiate between standard Digital I/O pins and Pulse Width Modulation (PWM) pins on Arduino UNO. [2 Marks]

The UNO has 14 digital pins, 6 of which support Pulse Width Modulation.

- **Standard Digital I/O Pins:** Handle strict binary states: HIGH (5V) or LOW (0V).
- **PWM Pins:** Marked with ~ (pins 3, 5, 6, 9, 10, 11); they toggle rapidly between 0V and 5V to simulate variable analog output, e.g., LED dimming or motor speed.

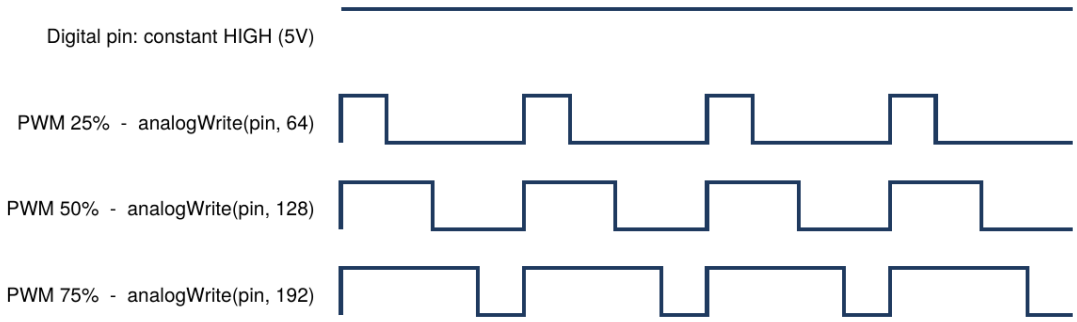


Fig 16: Digital output vs PWM duty cycles

Q18. Explain the primary function of the onboard voltage regulator on Arduino UNO. [2 Marks]

The voltage regulator is the power-management component of the Arduino UNO.

- **Input Voltage Stabilisation:** Steps 7-12V from the DC jack or VIN pin down to a stable 5V rail.
- **Hardware Protection:** Regulating fluctuating input protects the ATmega328P and sensor modules from over-voltage damage.

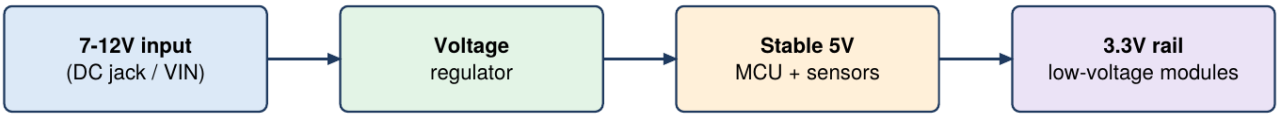


Fig 17: Voltage regulation

Q19. Define sensors and actuators in an IoT hardware ecosystem with relevant examples. [2 Marks]

Sensors and actuators are the two hardware components that connect embedded systems to the physical environment.

- **Sensors (Data Inputs):** Measure physical parameters and convert them to electrical signals, e.g., DHT11 (temperature/humidity) or PIR (motion).
- **Actuators (Action Outputs):** Turn electrical signals from the microcontroller into physical action, e.g., servo motors, 5V relay modules or smart LED lights.

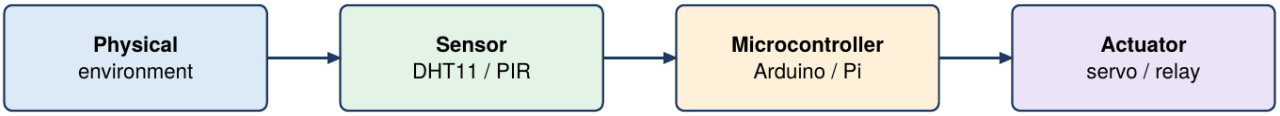


Fig 18: Sensor - controller - actuator chain

Q20. Define a Single-Board Computer (SBC) in the context of Raspberry Pi. [2 Marks]

A Single-Board Computer (SBC) is a complete, functional computer built on a single printed circuit board; the Raspberry Pi is one.

- **Complete Integrated Computer:** Includes CPU, graphics processor, RAM, storage interface, USB ports and network controllers.
- **OS Capability:** Unlike basic microcontroller boards, it can run a full desktop OS such as Linux and several applications at once.

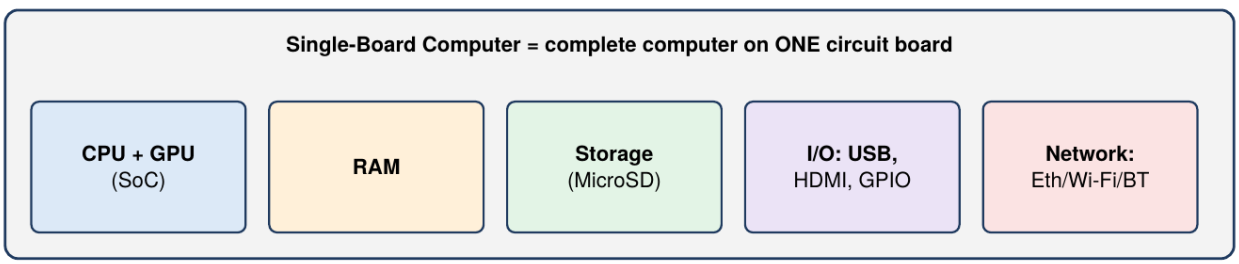


Fig 19: Components of an SBC

Q21. What is the functional requirement of the MicroSD card slot in Raspberry Pi hardware? [2 Marks]

The MicroSD slot is the onboard storage and boot interface on the underside of the Raspberry Pi board.

- **Operating System Boot Medium:** With no onboard flash storage, the card holds the bootloader partition and system files needed to boot Raspberry Pi OS.
- **Primary System Storage:** Stores installed software, Python scripts, user documents and database files.

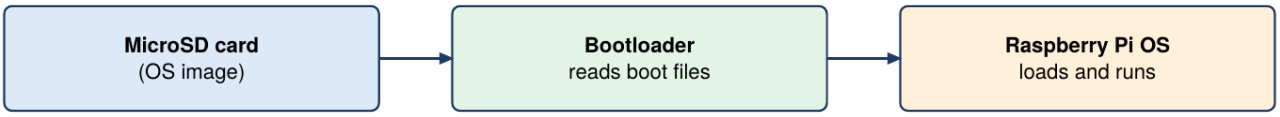


Fig 20: Boot from MicroSD

Q22. Differentiate between Camera Serial Interface (CSI) and Display Serial Interface (DSI) on Raspberry Pi. [2 Marks]

The Pi has two ribbon-cable connectors: CSI for cameras and DSI for displays.

- **Camera Serial Interface (CSI):** Connects official camera modules directly to the Broadcom processor for high-speed image/video input.
- **Display Serial Interface (DSI):** Connects official LCD panels through a 15-pin ribbon cable, sending raw video to a touchscreen.

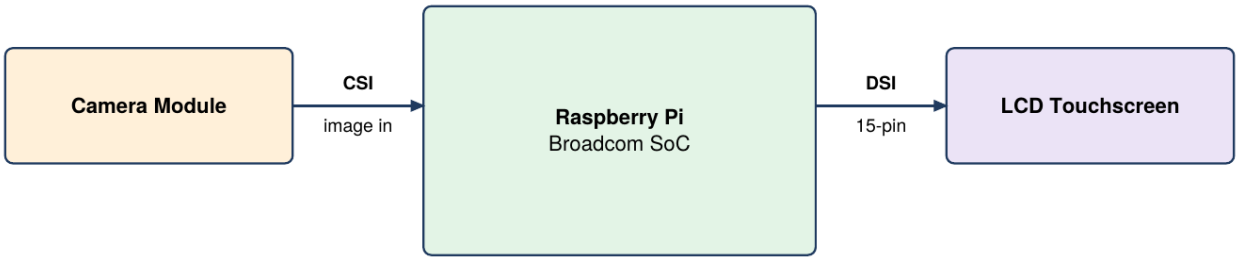


Fig 21: CSI (input) vs DSI (output)

Q23. What is the primary role of an IoT gateway in end-to-end data processing? [2 Marks]

An IoT gateway is the intermediate bridge between local devices and cloud application servers.

- **Data Aggregation and Preprocessing:** Collects telemetry from many low-power nodes, filters noise and reformats payloads before sending across the WAN.
- **Local Edge Computation:** Applies decision logic at the edge for fast responses to critical events without cloud latency.

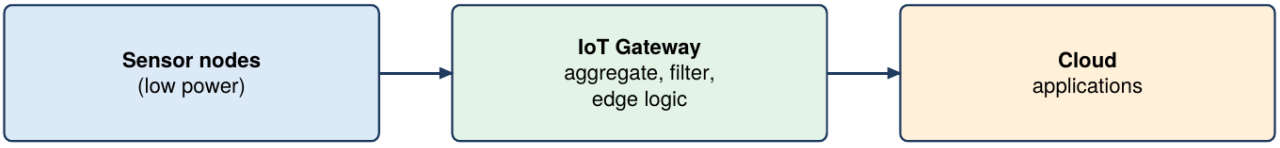


Fig 22: Gateway in end-to-end processing